# Tender Pre-Processing Service

Техническое задание

**Версия: 0.4** **Дата: 05.10.2026** **Автор: Arsenii Alhimovici**

## Содержание


1 Введение ..... 2
1.1 Назначение ..... 2
1.2 Зачем нужна система ..... 2
1.3 Место в общем процессе ..... 2
1.4 Термины ..... 2
2 Границы системы ..... 3
3 Роли ..... 5
4 Входные данные ..... 5
5 Требования ..... 5
5.1 Функциональные требования ..... 5
5.2 Бизнес-правила ..... 6
5.3 Нефункциональные требования ..... 7
6 Диаграмма вариантов использования ..... 7
7 Прецеденты ..... 8
8 Бизнес-процесс ..... 11
9 Динамика системы ..... 12
10 Данные ..... 15
11 Структура классов ..... 16
12 Интеграции и API ..... 16
12.1 REST API sourcing-service ..... 16
12.2 События Kafka ..... 17
12.3 Price Parser Service ..... 17
12.4 Tavily и DeepSeek ..... 17
13 Алгоритм подбора и оценки ..... 17
14 Технологический стек ..... 18
15 Допущения, ограничения и риски ..... 18

# 1 Введение

## 1.1 Назначение

**Tender Pre-Processing Service** (TPS) — веб-сервис предварительной обработки тендера. По рабочей документации (РД) в формате JSON он находит потенциальных поставщиков в заданном регионе, оценивает, какие позиции они могут закрыть, даёт оценку порядка стоимости по их прайс-листам и помогает разослать им запросы коммерческих предложений (КП).

## 1.2 Зачем нужна система

Когда компания выходит на новый тендер или новый рынок, у неё нет списка проверенных поставщиков. Кому писать и у кого запрашивать материалы, приходится выяснять вручную: искать компании в интернете, открывать сайты, смотреть ассортимент, выписывать контакты и рассылать письма. Это занимает дни и не даёт даже приблизительного понимания бюджета.

TPS закрывает самый ранний этап закупки — до получения КП:

- за минуты превращает РД в список потенциальных поставщиков с указанием, что именно они могут закрыть;
- показывает порядок стоимости по реальным прайс-листам, чтобы заранее понимать масштаб бюджета;
- сокращает ручную работу по рассылке: письма с нужными позициями формируются автоматически, менеджер только проверяет и отправляет.

## 1.3 Место в общем процессе

Сервис полностью автономен. Его результат — отправленные запросы КП. Ответы поставщики присылают на почту, после чего КП загружаются в основную систему анализа тендерных цен вручную; TPS в этой загрузке не участвует.

РД (JSON) ⟶ **TPS** ⟶ запросы КП (e-mail) ⟶ КП поставщиков ⟶ основная система анализа цен

*Рис. 1. Место TPS в процессе закупки*

## 1.4 Термины

| Термин | Определение |
|---|---|
| РД | Рабочая документация: перечень позиций (материалов и оборудования) с количествами. |
| Позиция | Одна строка РД: наименование, количество, характеристики, категория. |
| Категория | Группа позиций РД (поле `group`), например «Радиаторы отопления»; подкатегория — `group_ksr`. |
| КП | Коммерческое предложение поставщика с ценами на запрошенные позиции. |
| Кандидат | Потенциальный поставщик, найденный сервисом для конкретной задачи. |
| Покрытие | Какую долю позиций и каких категорий РД поставщик, по оценке сервиса, может поставить. |
| Маржа подрядчика | Наценка подрядчика, добавляемая к стоимости материалов; по умолчанию 20%. |
| Порядок стоимости | Ориентировочная итоговая сумма, достаточная для оценки масштаба, но не точная цена. |

## 2 Границы системы

**Внутри системы:**

- приём и проверка РД в формате JSON;
- группировка позиций по категориям;
- поиск потенциальных поставщиков и оценка покрытия (с помощью Tavily и DeepSeek);
- реестр поставщиков и их прайс-листов в собственной базе данных;
- расчёт оценки порядка стоимости;
- формирование, редактирование и отправка писем с запросом КП;
- веб-интерфейс менеджера.

**Снаружи системы (внешнее окружение):**

- **Tavily API** — поиск страниц в интернете;
- **DeepSeek API** — анализ найденных страниц (вызывается напрямую);
- **Price Parser Service** — существующий сервис, получающий прайс-листы с сайтов поставщиков;
- **SMTP-сервер** — отправка писем;
- **поставщики** — получатели писем, отвечают по почте;
- **основная система анализа тендерных цен** — принимает КП вне TPS.

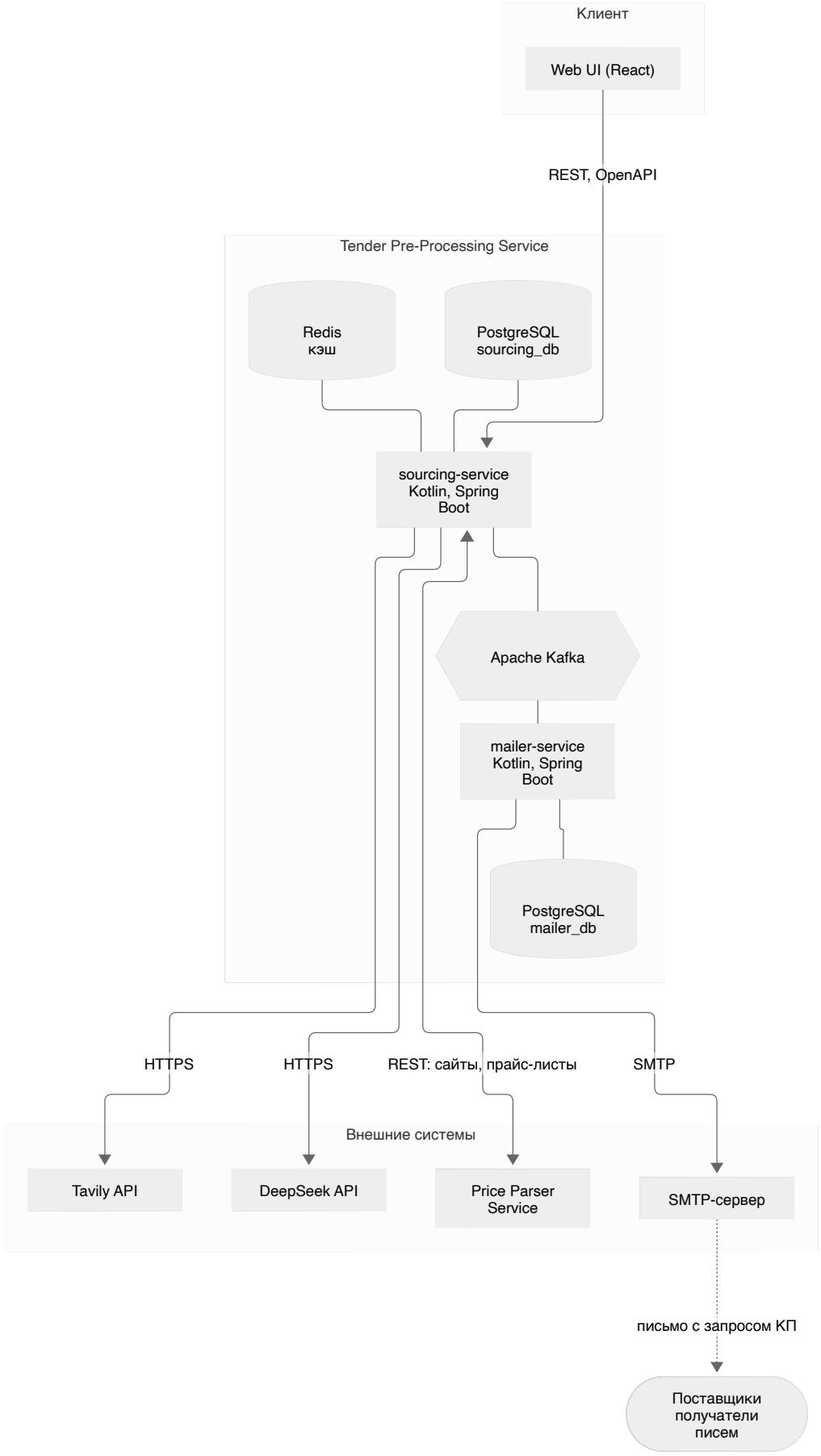



*Рис. 2. Архитектура системы*

Система состоит из двух микросервисов. **sourcing-service** — основной: принимает РД, ищет и оценивает поставщиков, считает стоимость, готовит письма и предоставляет единое REST API для интерфейса. **mailer-**

**service** — отправляет письма через SMTP и сообщает о статусах. Сервисы общаются через Apache Kafka; у каждого своя база PostgreSQL. Redis используется как кэш результатов внешних вызовов.

# 3 Роли

| Роль | Описание |
|---|---|
| Менеджер по закупкам | Единственная роль пользователя. Загружает РД, проверяет список поставщиков, при необходимости исправляет его, просматривает оценку, редактирует и отправляет письма. Работает в одном рабочем пространстве; авторизация в первой версии не требуется. |

Администратор и мультитенантность в первую версию не входят.

# 4 Входные данные

Входной файл — JSON-объект с массивом `products`. Для проекта проанализирован реальный пример РД: 352 позиции, 10 категорий (`group`), 32 подкатегории (`group_ksr`); цены в РД не указаны (`price = null`), валюта поля — `rub`. Регион, язык писем и маржа в файле не хранятся и задаются при загрузке.

| Поле позиции | Использование |
|---|---|
| `raw_name` | Наименование позиции; основа для текста письма и поиска по прайс-листу. |
| `group`, `group_ksr` | Категория и подкатегория; по ним строятся поисковые запросы и покрытие. |
| `quantity` | Количество; участвует в расчёте стоимости и попадает в письмо. |
| `item_index`, `detail_number` | Номер позиции и артикул; выводятся в письме. |
| `numeric_params`, `word_lists` | Характеристики (размеры, тип); используются для уточнения совпадений. |
| `supplier`, `price`, `currency` | Не обязательны; сохраняются для справки. |

# 5 Требования

## 5.1 Функциональные требования

| ID | Требование |
|---|---|
| FR-001 | Система должна принимать РД в формате JSON (объект с массивом `products`) и проверять его структуру; при ошибке сообщать, какое поле некорректно. |
| FR-002 | При создании задачи менеджер задаёт регион (RU, MD или EU), язык писем (ru, ro или en) и маржу подрядчика (по умолчанию 20%). |
| FR-003 | Система должна группировать позиции по категориям (`group`) и подкатегориям (`group_ksr`) и показывать число позиций в каждой. |
| FR-004 | Система должна формировать поисковые запросы по каждой категории с учётом региона и языка региона. |
| FR-005 | Система должна искать потенциальных поставщиков через Tavily. |

| ID | Требование |
|---|---|
| FR-006 | Система должна анализировать найденные страницы через DeepSeek: определять, является ли компания поставщиком, извлекать название, сайт, e-mail, телефон и оценивать, какие категории и сколько позиций она покрывает. |
| FR-007 | Система должна объединять дубликаты (по домену сайта) и агрегировать покрытие поставщика по категориям. |
| FR-008 | Система должна показывать список поставщиков: название, сайт, контакты, покрываемые категории, примерное число покрываемых позиций, среднюю цену по прайс-листу (если он получен) и уровень уверенности. |
| FR-009 | Система должна передавать сайты найденных поставщиков в Price Parser Service, получать прайс-листы в JSON и сохранять их. |
| FR-010 | Менеджер может удалить поставщика из списка, вернуть его и добавить поставщика вручную (название, сайт, e-mail). |
| FR-011 | Система должна рассчитывать оценку порядка стоимости по подтверждённым поставщикам и показывать базовую сумму, маржу подрядчика отдельной строкой, итог, диапазон и уровень уверенности; при изменении списка оценка пересчитывается. |
| FR-012 | Суммы отображаются в USD всегда и дополнительно в валюте региона: RU — RUB, MD — MDL, EU — EUR. |
| FR-013 | Система должна формировать для каждого выбранного поставщика черновик письма-запроса КП на выбранном языке: название тендера, перечень позиций категорий поставщика с количествами, срок ответа. |
| FR-014 | Менеджер может изменить тему и текст каждого письма до отправки. |
| FR-015 | Система должна отправлять письма через SMTP: одно письмо или все выбранные одной кнопкой. |
| FR-016 | Система должна показывать статус каждого письма: черновик, в очереди, отправлено, ошибка. |
| FR-017 | Система должна кэшировать результаты поиска, анализа и прайс-листов в Redis, чтобы повторная обработка тех же запросов не обращалась к внешним сервисам. |
| FR-018 | Система должна сохранять историю задач и позволять открыть результат прошлой обработки. |
| FR-019 | Система должна показывать прогресс обработки по этапам. |

## 5.2 Бизнес-правила

| ID | Правило |
|---|---|
| BR-001 | Валюты отображения по региону: RU — USD и RUB; MD — USD и MDL; EU — USD и EUR. Расчёты выполняются в USD. |
| BR-002 | Итог = базовая стоимость + маржа; маржа = базовая стоимость × процент маржи и выводится отдельной строкой. |
| BR-003 | Уровень уверенности оценки определяется долей позиций, для которых найдена цена: менее 40% — низкий, 40–70% — средний, более 70% — высокий. |
| BR-004 | В письмо попадают только позиции тех категорий, которые поставщик, по оценке, покрывает. |

| ID | Правило |
|---|---|
| BR-005 | Оценка стоимости есть ориентир порядка величины и не является ценовым предложением. |

### 5.3 Нефункциональные требования

| ID | Требование |
|---|---|
| PR-001 | Поиск, анализ и расчёт оценки для РД до 400 позиций выполняются не более чем за 5 минут (в 95% запусков). |
| PR-002 | Просмотр списков и пересчёт оценки после правки списка поставщиков выполняются не более чем за 3 секунды. |
| REL-001 | При недоступности внешнего сервиса выполняется до 3 повторных попыток с нарастающей задержкой; задача не прекращается целиком, пользователь видит частичный результат и причину сбоя. |
| REL-002 | Письмо не отправляется дважды при повторной доставке события из Kafka (идемпотентность по идентификатору письма). |
| COST-001 | Число запросов к Tavily и DeepSeek на одну задачу ограничено настраиваемым лимитом; повторные запросы берутся из кэша. |
| SEC-001 | Ключи API и учётные данные SMTP хранятся только в переменных окружения или секретах и не попадают в репозиторий и журналы. |
| SEC-002 | Размер загружаемого файла ограничен (по умолчанию 10 МБ); входной JSON проверяется до обработки. |
| I18N-001 | Тексты писем поддерживаются на русском, румынском и английском языках. |
| MAINT-001 | REST API описывается в OpenAPI (contract-first); серверные интерфейсы и клиент интерфейса генерируются из спецификации. |
| MAINT-002 | Схема базы данных создаётся и изменяется только миграциями Liquibase. |
| MAINT-003 | На каждый push и pull request в GitHub Actions выполняются сборка, тесты и статический анализ кода; при ошибках слияние запрещено. |
| MAINT-004 | Вся система запускается локально одной командой через Docker Compose. |

## 6 Диаграмма вариантов использования

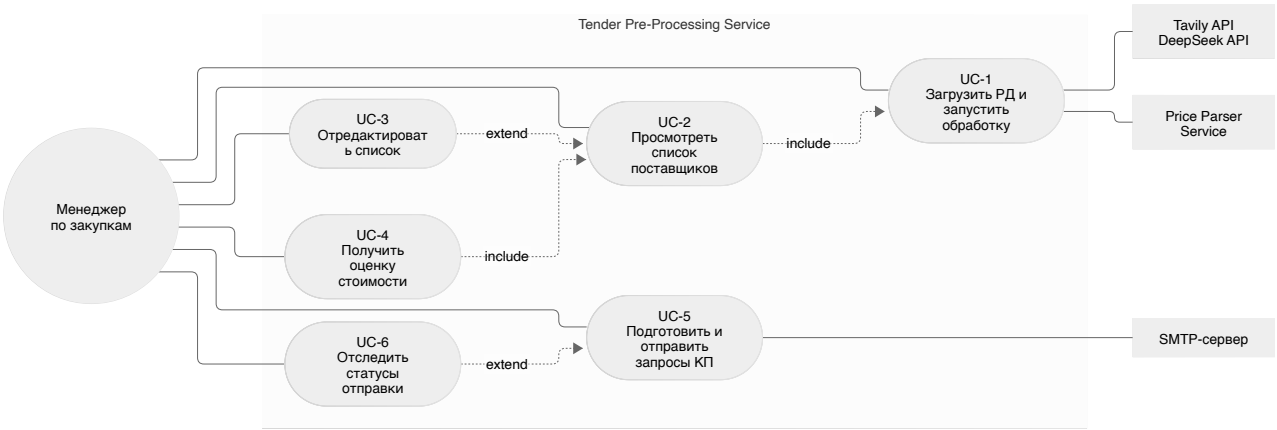



*Рис. 3. Диаграмма вариантов использования*

# 7 Прецеденты

**Прецедент UC-1: Загрузка РД и запуск обработки**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер загружает РД, выбирает регион, язык писем и маржу; система запускает поиск и анализ поставщиков.
**Предусловия:** Менеджер открыл страницу создания задачи; файл РД доступен.
**Основной поток:**
1. Менеджер выбирает файл РД в формате JSON.
2. Менеджер указывает название тендера, регион, язык писем и маржу.
3. Система проверяет структуру файла и ограничения по размеру.
4. Система создаёт задачу, группирует позиции по категориям и запускает обработку.
5. Система показывает прогресс по этапам (поиск, анализ, получение цен, расчёт).
6. По завершении система переводит задачу в статус «Готово к проверке».

**Альтернативные потоки:**
- 3а. Файл некорректен: система сообщает, какое поле не прошло проверку; менеджер исправляет файл и повторяет загрузку.
- 5а. Внешний сервис недоступен: система повторяет запрос; при неудаче продолжает с частичными данными и отмечает проблему.

**Постусловия:** Задача создана; список поставщиков и оценка готовы либо помечены как частичные.

**Прецедент UC-2: Просмотр списка поставщиков**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер просматривает найденных поставщиков и то, что они покрывают.
**Предусловия:** Задача в статусе «Готово к проверке».
**Основной поток:**
1. Менеджер открывает задачу.
2. Система показывает список: название, сайт, контакты, категории, примерное число покрываемых позиций, средняя цена, уверенность.
3. Менеджер открывает карточку поставщика и видит источник, по которому он найден.

**Альтернативные потоки:**
- 2а. Поставщиков не найдено: система сообщает об этом и предлагает добавить поставщиков вручную.

**Постусловия:** Менеджер получил картину покрытия РД.

**Прецедент UC-3: Редактирование списка поставщиков**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер убирает неподходящих поставщиков, возвращает удалённых и добавляет своих.
**Предусловия:** Список поставщиков сформирован.
**Основной поток:**
1. Менеджер удаляет поставщика из списка.
2. Система помечает кандидата как отклонённого и пересчитывает оценку.
3. Менеджер при необходимости добавляет поставщика вручную: название, сайт, e-mail.
4. Система передаёт сайт нового поставщика в Price Parser Service и обновляет оценку.

**Альтернативные потоки:**
- 3а. Сайт или e-mail указан некорректно: система сообщает об ошибке; менеджер исправляет данные.

**Постусловия:** Список поставщиков и оценка соответствуют решению менеджера.

**Прецедент UC-4: Получение оценки порядка стоимости**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер видит ориентировочную итоговую стоимость по подтверждённым поставщикам.
**Предусловия:** Есть хотя бы один подтверждённый поставщик.
**Основной поток:**
1. Менеджер открывает раздел «Оценка».
2. Система показывает базовую сумму, маржу подрядчика отдельной строкой, итог, диапазон и уровень уверенности.
3. Суммы выводятся в USD и в валюте региона.
4. Менеджер может изменить процент маржи; система пересчитывает итог.

**Альтернативные потоки:**
- 2а. Цены найдены не для всех позиций: система показывает долю позиций с ценой и снижает уровень уверенности.

**Постусловия:** Менеджер получил оценку порядка стоимости.

**Прецедент UC-5: Подготовка и отправка запросов КП**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер проверяет письма с перечнем нужных позиций и отправляет их поставщикам.
**Предусловия:** Подтверждённые поставщики выбраны; у каждого указан e-mail.
**Основной поток:**
1. Менеджер открывает раздел «Письма».
2. Система формирует черновики: тема, текст и перечень позиций категорий поставщика с количествами.
3. Менеджер при необходимости редактирует текст.
4. Менеджер нажимает «Отправить» для одного письма или для всех выбранных.
5. Система ставит письма в очередь; mailer-service отправляет их по SMTP.

**Альтернативные потоки:**
- 2a. У поставщика нет e-mail: система помечает письмо как неотправляемое и предлагает указать адрес.
- 5a. SMTP недоступен: mailer-service повторяет отправку до 3 раз, затем фиксирует ошибку.

**Постусловия:** Письма отправлены или имеют статус ошибки.

**Прецедент UC-6: Отслеживание статусов отправки**
**Главный актёр:** Менеджер по закупкам
**Краткое описание:** Менеджер видит, какие письма отправлены, а какие нет.
**Предусловия:** Письма поставлены в очередь отправки.
**Основной поток:**
1. Менеджер открывает раздел «Письма».
2. Система показывает статус каждого письма: в очереди, отправлено, ошибка.
3. Для письма с ошибкой система показывает причину; менеджер может повторить отправку.

**Альтернативные потоки:**
- 2a. Статус ещё не обновился: система показывает «в очереди» и обновляет данные при следующем запросе.

**Постусловия:** Менеджер знает, кому запрос ушёл, а кому нужно повторить.

# 8 Бизнес-процесс

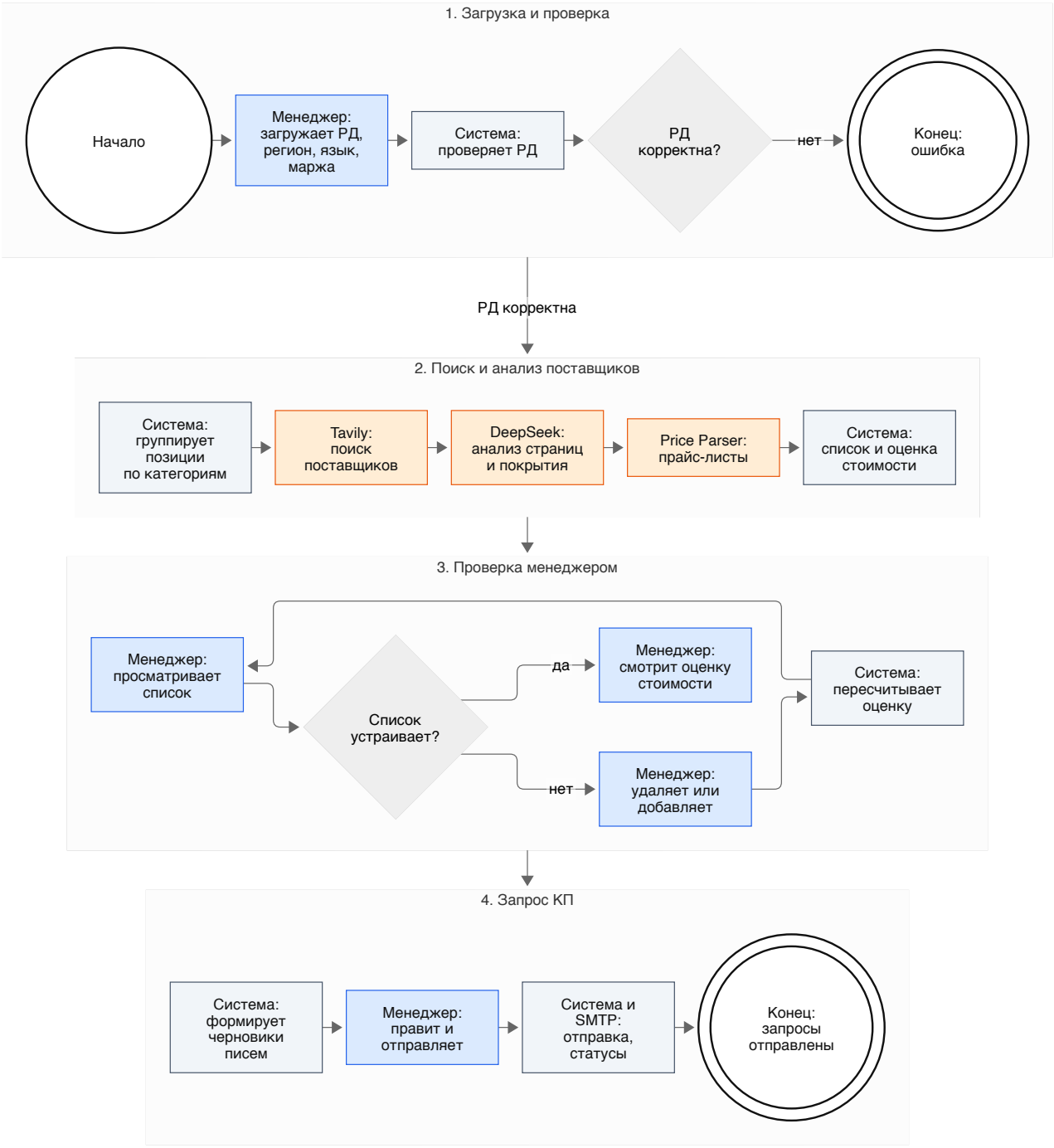



*Рис. 4. Бизнес-процесс «Подбор поставщиков и запрос КП»*

Диаграмма использует элементы BPMN: события начала и конца (круги), задачи (прямоугольники) и шлюзы (ромбы). Участники процесса показаны цветом: синий — менеджер, серый — система, оранжевый — внешние сервисы.

# 9 Динамика системы

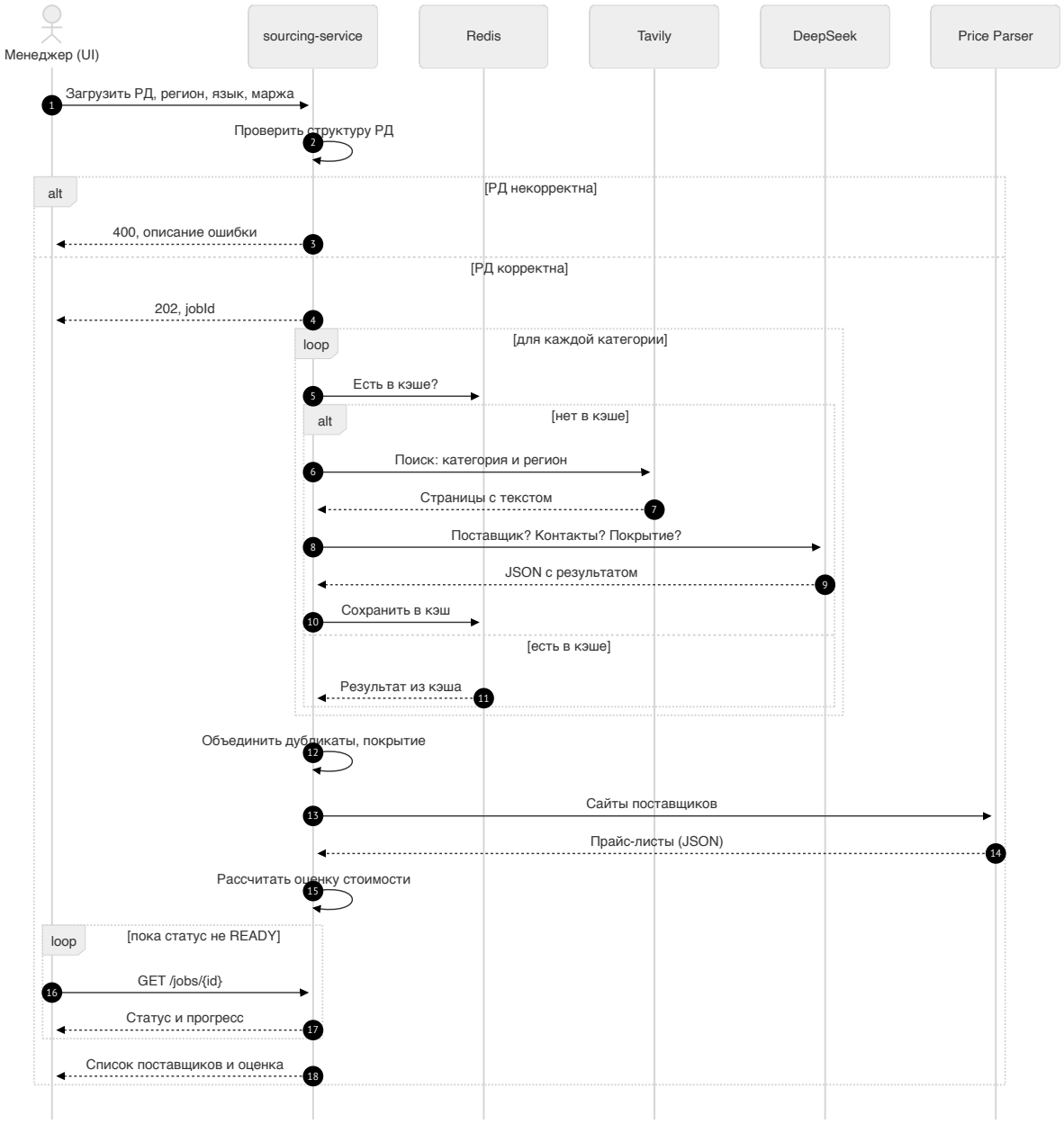



*Рис. 5. Последовательность: обработка РД и подбор поставщиков*

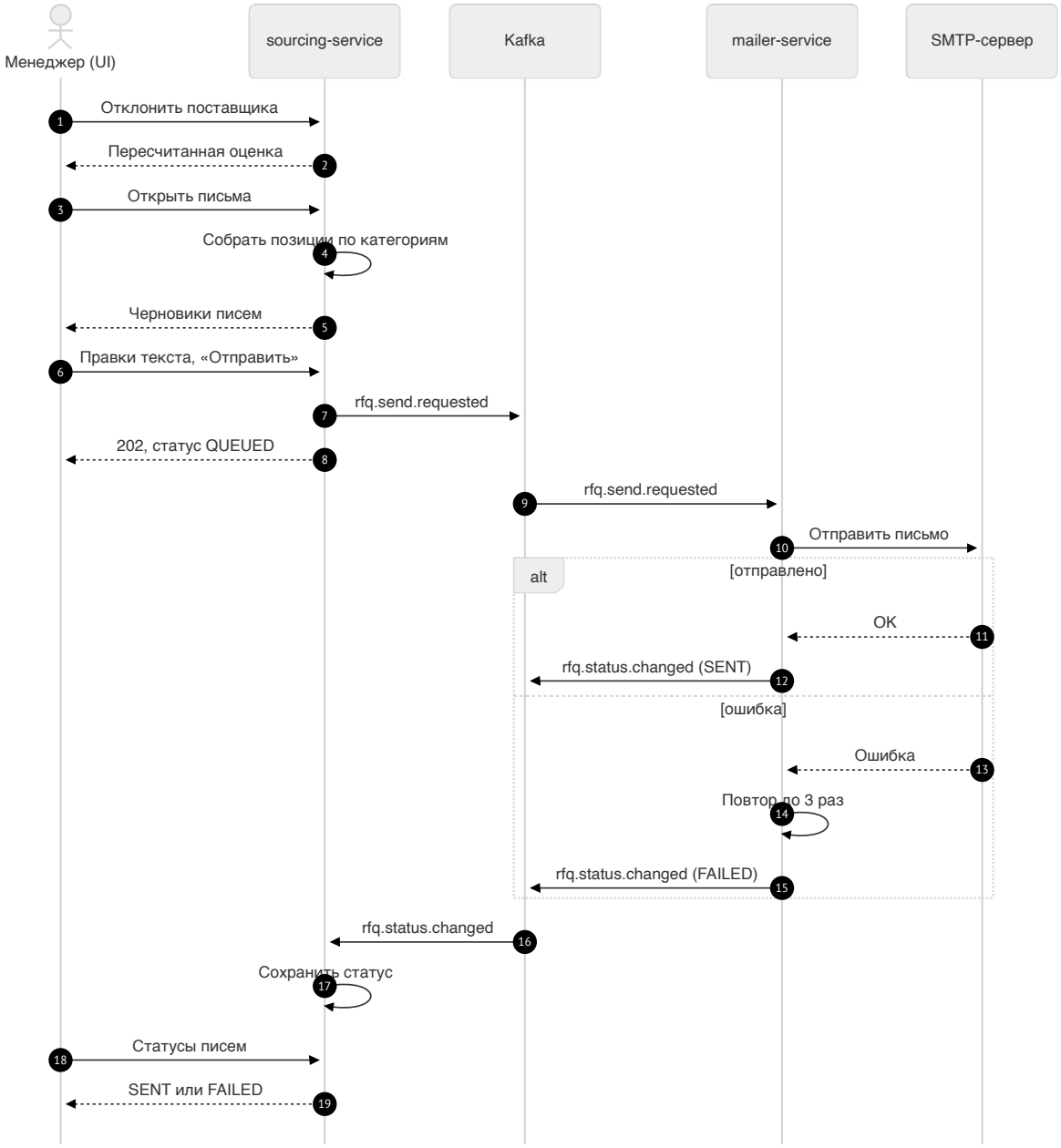



*Рис. 6. Последовательность: правка списка и отправка запросов КП*

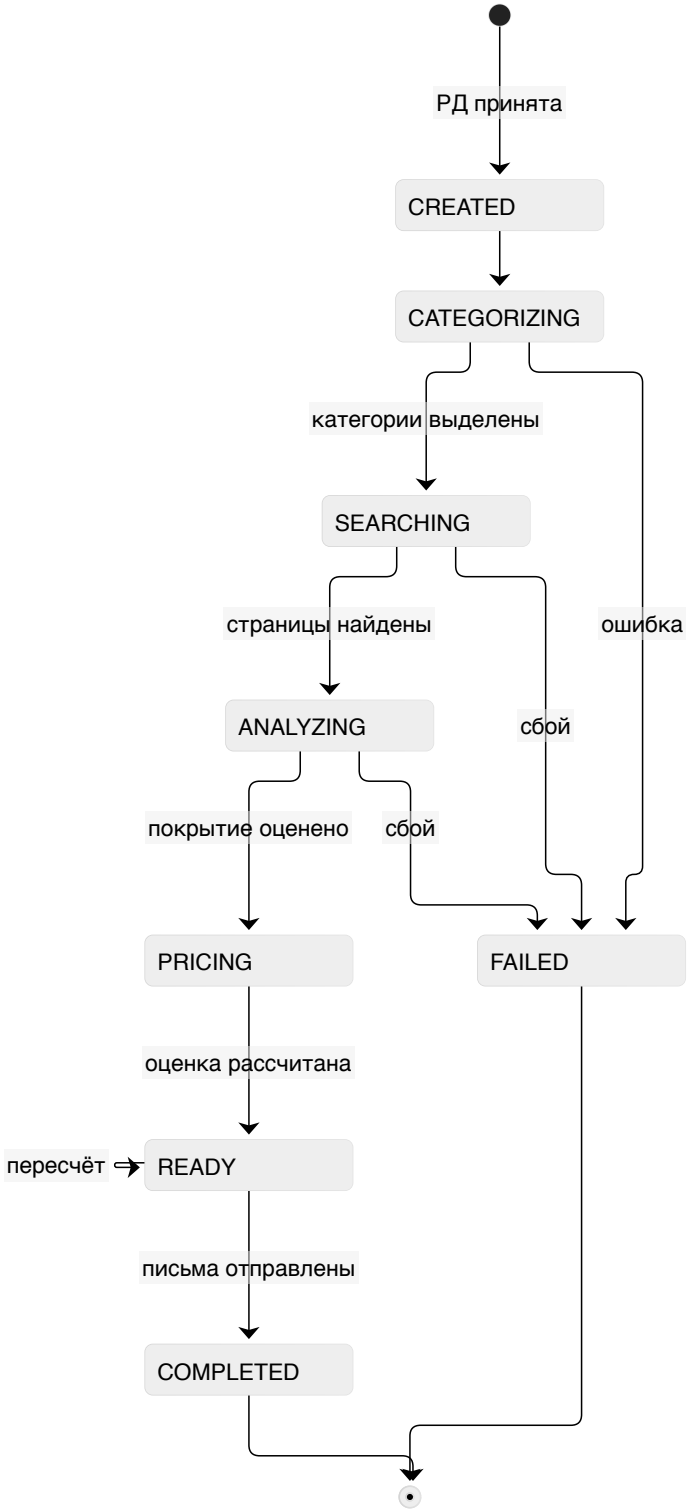



*Рис. 7. Состояния задачи обработки (READY — «готово к проверке», COMPLETED — письма отправлены)*

# 10 Данные

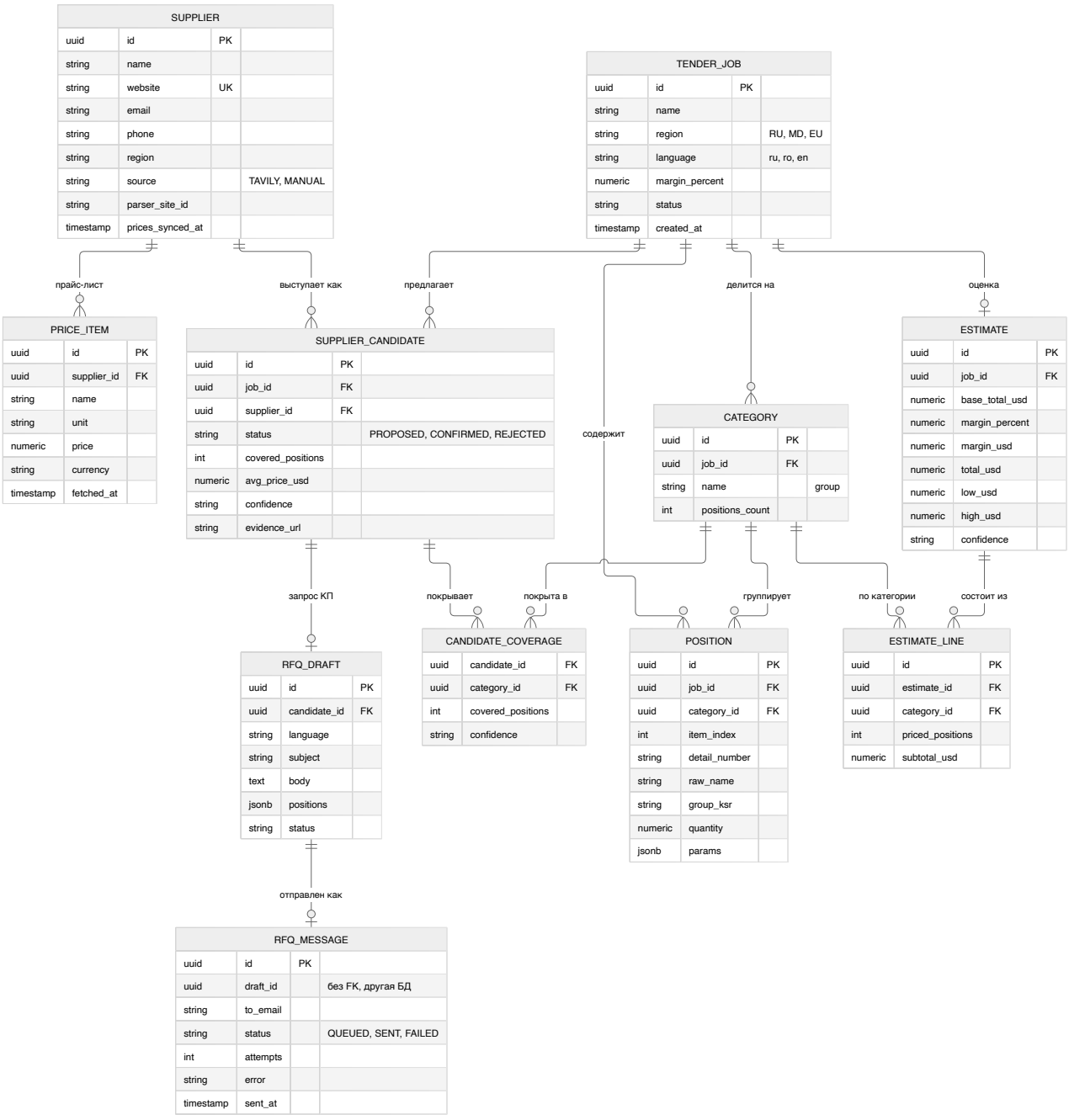



*Рис. 8. ER-диаграмма*

Базы данных двух сервисов независимы. Таблицы **RFQ_DRAFT** и **RFQ_MESSAGE** связаны по идентификатору без внешнего ключа, так как находятся в разных базах (sourcing_db и mailer_db). Поставщик (**SUPPLIER**) существует независимо от задач и накапливается в реестре, что позволяет повторно использовать найденных поставщиков и их прайс-листы в следующих тендерах. Курсы валют для отображения (USD в RUB, MDL, EUR) хранятся в справочнике.

# 11 Структура классов

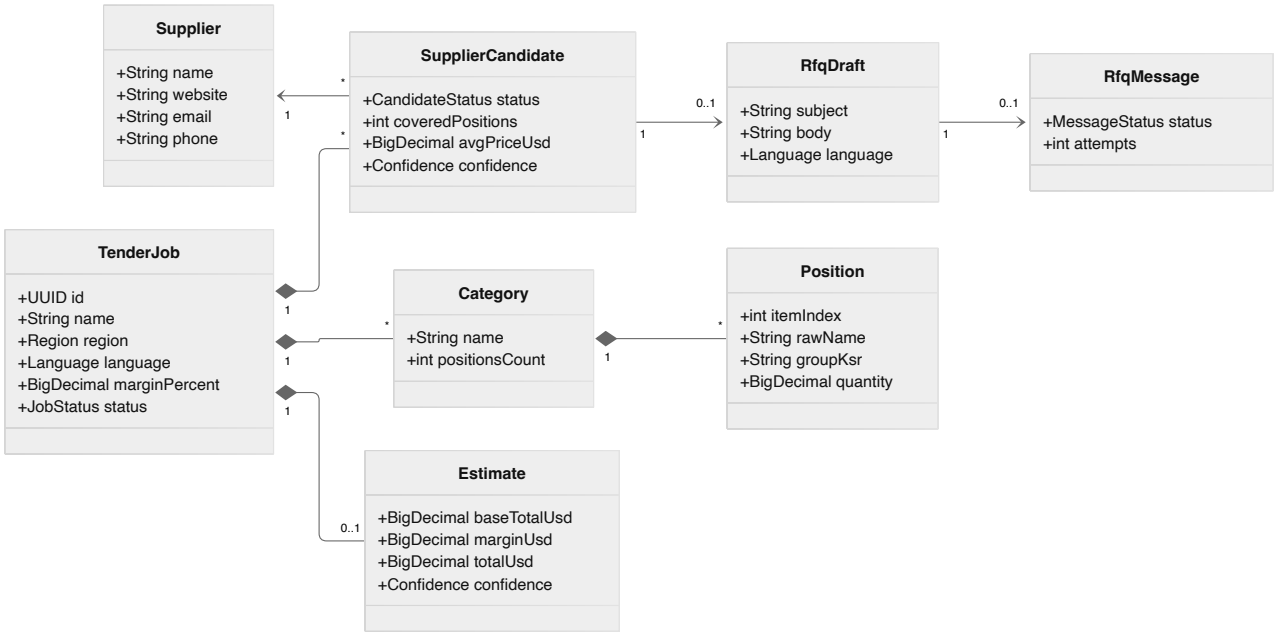



*Рис. 9. Диаграмма классов: доменная модель*

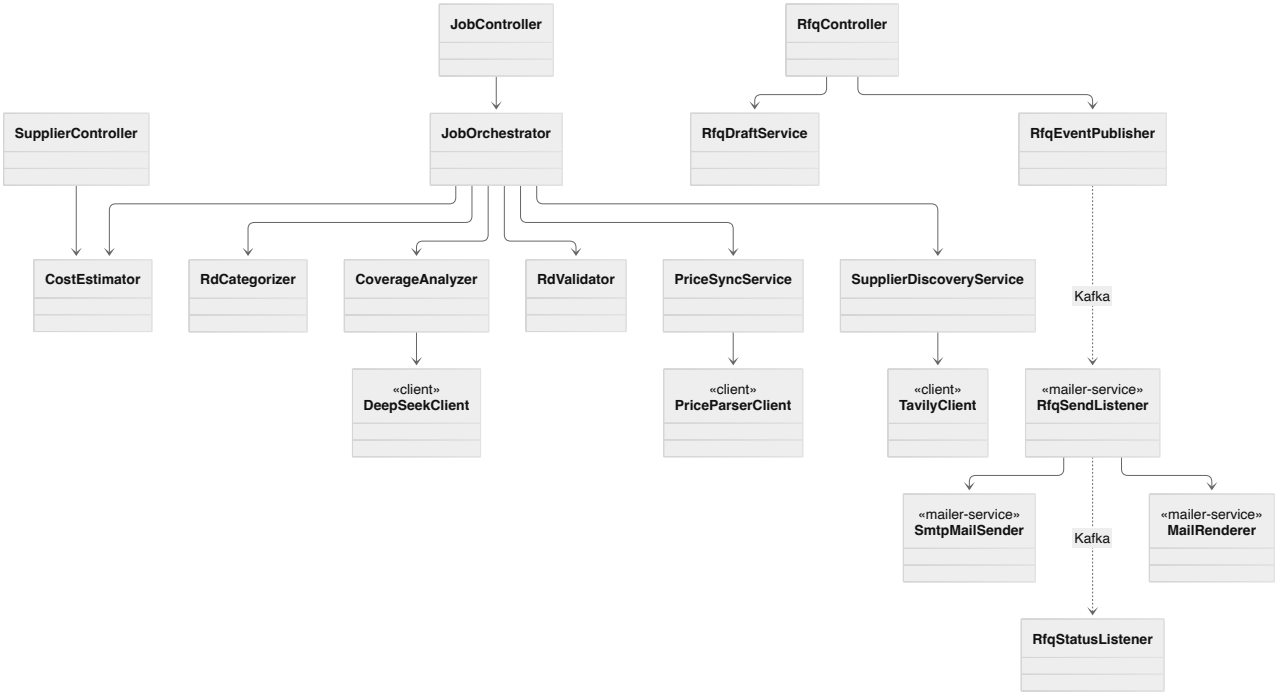



*Рис. 10. Диаграмма классов: сервисы, клиенты и обработчики событий*

# 12 Интеграции и API

## 12.1 REST API sourcing-service

Единая точка входа для интерфейса; описывается в OpenAPI. Основные методы:

| Метод | Путь | Назначение |
|---|---|---|
| POST | `/api/v1/jobs` | Загрузить РД (multipart) с регионом, языком и маржой; создать задачу, ответ 202. |

| Метод | Путь | Назначение |
|---|---|---|
| GET | `/api/v1/jobs` | Список задач. |
| GET | `/api/v1/jobs/{id}` | Статус и прогресс задачи. |
| GET | `/api/v1/jobs/{id}/categories` | Категории и число позиций. |
| GET | `/api/v1/jobs/{id}/suppliers` | Список поставщиков с покрытием. |
| POST | `/api/v1/jobs/{id}/suppliers` | Добавить поставщика вручную. |
| PATCH | `/api/v1/jobs/{id}/suppliers/{cid}` | Подтвердить или отклонить поставщика. |
| GET | `/api/v1/jobs/{id}/estimate` | Оценка стоимости; параметр `margin`. |
| GET | `/api/v1/jobs/{id}/rfq-drafts` | Черновики писем. |
| PUT | `/api/v1/rfq-drafts/{id}` | Изменить тему и текст письма. |
| POST | `/api/v1/jobs/{id}/rfq/send` | Отправить выбранные письма. |
| GET | `/api/v1/jobs/{id}/rfq/messages` | Статусы писем. |

### 12.2 События Kafka

| Топик | Направление | Содержимое |
|---|---|---|
| `rfq.send.requested` | sourcing → mailer | Идентификатор письма, получатель, тема, текст, язык. |
| `rfq.status.changed` | mailer → sourcing | Идентификатор письма, статус (SENT, FAILED), число попыток, причина ошибки. |

### 12.3 Price Parser Service

Существующий внешний сервис. TPS передаёт ему сайты поставщиков и получает прайс-листы. Формат передачи сайтов (согласован в первой версии):

```
{
  "site_ids": ["string"],
  "sites": [
    { "id": "string", "site_name": "string", "url": "string",
      "supplier_label": "string", "is_active": true }
  ]
}
```

Формат ответа с прайс-листами (позиции с наименованием, единицей, ценой и валютой) уточняется; TPS изолирует его в клиенте `PriceParserClient`, поэтому изменение формата не затрагивает остальную логику.

### 12.4 Tavily и DeepSeek

**Tavily** возвращает страницы и их текст по поисковому запросу. **DeepSeek** вызывается напрямую по HTTPS и по тексту страницы возвращает структурированный JSON: признак поставщика, название, сайт, контакты, покрываемые категории, оценка числа позиций, уверенность, ссылка-подтверждение.

## 13 Алгоритм подбора и оценки

1. **Группировка.** Позиции РД делятся на категории по `group`, внутри — по `group_ksr`.
2. **Запросы.** Для каждой категории формируется 2–3 запроса на языке региона вида «категория, поставщик, регион»; число запросов и результатов ограничено настройками.
3. **Поиск.** Tavily возвращает страницы с текстом; результаты кэшируются в Redis.

4. **Анализ.** DeepSeek оценивает каждую страницу: поставщик ли компания, какие подкатегории она покрывает и сколько позиций примерно.
5. **Объединение.** Кандидаты с одним доменом объединяются; покрытие суммируется по категориям, берётся максимальная уверенность.
6. **Прайс-листы.** Сайты кандидатов передаются в Price Parser Service. Если прайс-лист получен, покрытие уточняется: позиция считается покрытой, если в прайс-листе есть достаточно близкая по тексту позиция.
7. **Оценка цены позиции.** Цена берётся как медиана цен ближайших позиций из прайс-листов подтверждённых поставщиков (в USD); если близких позиций нет — средняя цена по категории; если и её нет, позиция остаётся без цены.
8. **Итог.** Базовая стоимость равна сумме произведений количества на цену по позициям с ценой. Маржа равна базовой стоимости, умноженной на процент маржи. Итог равен сумме базовой стоимости и маржи. Диапазон строится по нижней и верхней квартилям цен. Уверенность определяется по BR-003.

Пороги и лимиты являются настраиваемыми параметрами; значения по умолчанию уточняются при тестировании на реальной РД.

## 14 Технологический стек

| **Область** | **Технологии** |
|---|---|
| Интерфейс | React, Nginx; клиент API генерируется из OpenAPI |
| Серверная часть | Kotlin, Spring Boot (Web, Data JPA, Kafka, Mail), корутины для параллельных запросов к внешним сервисам |
| Данные | PostgreSQL (отдельная база у каждого сервиса), Liquibase для миграций, Redis для кэша |
| Обмен сообщениями | Apache Kafka |
| API и контракты | OpenAPI 3 (contract-first), openapi-generator |
| Внешние сервисы | Tavily API, DeepSeek API (напрямую/через openrouter), Price Parser Service, SMTP |
| Развёртывание | Docker, Docker Compose |
| Качество | Kotest, MockK, Testcontainers, detekt и ktlint, GitHub Actions |

## 15 Допущения, ограничения и риски

- Результаты поиска и оценки покрытия получены с помощью LLM и приблизительны; поэтому менеджер подтверждает список перед рассылкой, а в карточке поставщика показывается ссылка-источник.
- Оценка стоимости основана на прайс-листах, а не на КП; прайс-цены обычно выше цен КП, поэтому оценка даёт порядок величины, а не цену.
- Формат ответа Price Parser Service с прайс-листами ещё не окончательный; интеграция изолирована в отдельном клиенте.
- Курсы валют задаются в справочнике; источник и частота обновления определяются на этапе реализации.
- Не все сайты поставщиков могут быть обработаны парсером; такие поставщики остаются в списке без цен.
- Рассылка писем должна соответствовать требованиям закона о персональных данных и содержать возможность отказа от дальнейших писем.
- Не входит в первую версию: авторизация, роль администратора, несколько рабочих пространств, приём ответов с КП, интеграция с основной системой, нагрузочное тестирование.